```
D2D1_POINT_2F DeCasteljau(const std::vector<D2D1_POINT_2F>& controlPoints, float t) {
    std::vector<D2D1_POINT_2F> points = controlPoints;  // 复制控制点
    int n = points.size();

    // 递推计算: 从第1层到第n-1层
    for (int r = 1; r < n; r++) {
        // 每层计算n-r个点
        for (int i = 0; i < n - r; i++) {
            // 线性插值: P_i = (1-t)*P_i + t*P_(i+1)
            points[i].x = (1 - t) * points[i].x + t * points[i + 1].x;
            points[i].y = (1 - t) * points[i].y + t * points[i + 1].y;
        }
    }
    // 最终结果存储在points[0]
    return points[0];
}

/**
 * 绘制Bezier曲线
 * @param pRT 渲染目标
 * @param pBrush 画刷
 * @param controlPoints 控制点数组
 */
void DrawBezierCurve(ID2D1RenderTarget* pRT, ID2D1SolidColorBrush* pBrush,
                     const std::vector<D2D1_POINT_2F>& controlPoints) {
    if (controlPoints.size() < 2) return;  // 至少需要2个控制点

    const int samples = 100;  // 采样点数量
    std::vector<D2D1_POINT_2F> curvePoints;

    // 对参数t进行采样，计算曲线上的点
    for (int i = 0; i <= samples; i++) {
        float t = (float)i / samples;  // t从0到1
        curvePoints.push_back(DeCasteljau(controlPoints, t));
    }
```